# A Streaming BF16 Self-Attention Accelerator with RoPE, GQA, and a Time-Shared Systolic MAC Array

*İTÜ YONGA: Yonga Attention Engine (YAE) Project Team*
*Belinay Güler, Can Ertürk, M. Taha Aydemir, Hasan Sancak*

*Department of Electronics and Communication Engineering, Istanbul Technical University, Istanbul, Turkey*

***Abstract:*** ***This report presents the design of a SystemVerilog RTL accelerator that computes the self-attention block of transformer models in hardware. The core implements a streaming, AXI-Stream-based datapath in bfloat16 (BF16) arithmetic, and integrates Rotary Positional Encoding (RoPE), Grouped Query Attention (GQA), and a time-shared systolic processing-element (PE) array that performs both the $Q \cdot K^T$ similarity computation and the score·V reduction on the same physical hardware. Resource efficiency is achieved through ping-pong double buffering, valid/ready handshaking throughout, and arbitrated sharing of the most expensive multiply-accumulate (MAC) resource. The design comprises more than fifteen RTL modules, each validated by a standalone, self-checking testbench. All seven testbenches pass, and post-implementation synthesis on a Xilinx Artix-7 device measures 1,486 LUTs, 510 flip-flops, and zero DSP or Block RAM usage. A full end-to-end, top-level simulation of the assembled pipeline has since been run to completion without hanging or losing data, after four additional integration bugs surfaced and were fixed. Remaining work is a golden-model numerical cross-check of RoPE, a weight-loading handshake, and correcting softmax's row-normalization boundary, currently computed once per full score tile rather than once per row.***

***Index Terms:*** *Self-attention, transformer accelerator, RTL, SystemVerilog, bfloat16, RoPE, Grouped Query Attention, systolic array, FPGA.*

## I. INTRODUCTION

Modern language models read a sentence by deciding, for every word, how much attention to pay to every other word around it, the way a reader leans on earlier context to resolve a pronoun or to weigh a relevant clause more heavily than a filler one. Transformer-based models implement this idea as a specific mechanism called self-attention, which is computationally dominated by matrix multiplications and a row-wise softmax normalization. Mapping this mechanism to dedicated hardware requires careful management of arithmetic precision, data movement, and resource reuse. This work implements an end-to-end streaming attention core that takes a token vector through Q/K/V projection, RoPE positional encoding, GQA head mapping, $Q \cdot K^T$ similarity computation, scaling and masking, softmax normalization, and finally the score·V product, all communicated between modules using an AXI-Stream-like valid/ready handshake.

A central design goal was resource efficiency. Rather than instantiating separate hardware for the two large matrix-multiplication stages of attention, the design shares a single systolic PE array, referred to in this report as Module A, between the $Q \cdot K^T$ computation and the score·V computation, distinguishing the two data streams with a tag bit and arbitrating access at the input.

## II. SYSTEM ARCHITECTURE

Fig. 1 shows the top-level data flow. Input tokens arrive over the AXI interface and are written into a double-buffered memory (dbuf), which allows new data to be loaded while computation proceeds on the current buffer. Data then passes sequentially through QKV projection, RoPE, and GQA head mapping. In the second stage, the $Q \cdot K^T$ array produces similarity scores, which are scaled, masked, and normalized by the softmax unit before the score·V operation produces the final output, which is returned to the AXI interface. As noted above, the $Q \cdot K^T$ array and the score·V unit share the same physical MAC hardware (Module A). An arbiter/router pair manages access to this shared resource using a one-bit tag carried alongside the data.

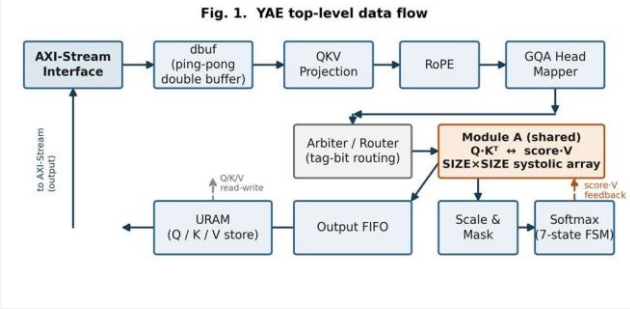


*Fig. 1. Top-level system block diagram showing the AXI-Stream datapath, double-buffered memory, QKV/RoPE/GQA front end, and the shared $Q \cdot K^T$ / score·V array with its arbiter and router.*

## III. BF16 ARITHMETIC CORE

All datapath computation uses the 16-bit bfloat16 (BF16) format, which reduces memory footprint and interconnect bandwidth relative to IEEE-754 float32 while retaining float32's dynamic range. The arithmetic core provides four basic units: an adder, a multiplier, and two format converters. Both the adder and multiplier are implemented as two-stage pipelines rather than single-cycle combinational blocks, so every module that consumes their results is built around the valid/ready handshake instead of a fixed latency assumption. To support the shared-hardware scheme described in Section II, all data on the internal bus is carried as 17 bits: a 16-bit BF16 value plus a 1-bit tag identifying the data's source stream.

## IV. QKV PROJECTION

The QKV projection module performs a matrix-vector multiply-accumulate between the incoming token vector and a weight matrix. A single parameterized module is instantiated four times, for the Q, K, V, and output projections, each configured with different weights. Because the design supports GQA, the output dimensions of the K and V projections are configured to be smaller than that of Q, reflecting the smaller number of key/value heads relative to query heads.

## V. ROTARY POSITIONAL ENCODING (ROPE)

RoPE embeds positional information into the Q and K vectors by applying a position-dependent rotation using sine and cosine coefficients. The implementation uses the "rotate-half" pairing convention, in which element i is paired with element i + D_MODEL/2 rather than with an adjacent element. This pairing choice is currently an assumption rather than a confirmed specification, since it was not made explicit in the project plan, and it is flagged for verification once a golden-model reference script is available (see Section IX).

## VI. GROUPED QUERY ATTENTION (GQA) MAPPER

In Grouped Query Attention, the number of query heads exceeds the number of key/value heads, and groups of query heads share a common K/V pair. In the current configuration, eight Q heads map to two KV heads. The GQA mapper does not move or duplicate data. It generates the index that determines which KV head a given Q head should use, deferring the actual head selection to downstream modules.

## VII. SHARED $Q \cdot K^T$ / SCORE·V SYSTOLIC ARRAY

Module A is a SIZE × SIZE output-stationary systolic PE array. Query and key elements are streamed in serially along the array's depth dimension. Once all elements for a given tile have been accumulated, the array switches to a parallel read-out phase. Because this array is time-shared between the $Q \cdot K^T$ similarity computation and the score·V reduction, an input arbiter governs access: priority is given to score·V requests arriving from the softmax stage, with new $Q \cdot K^T$ inputs accepted afterward. This ordering keeps the softmax-to-output path from stalling behind newly arriving queries.

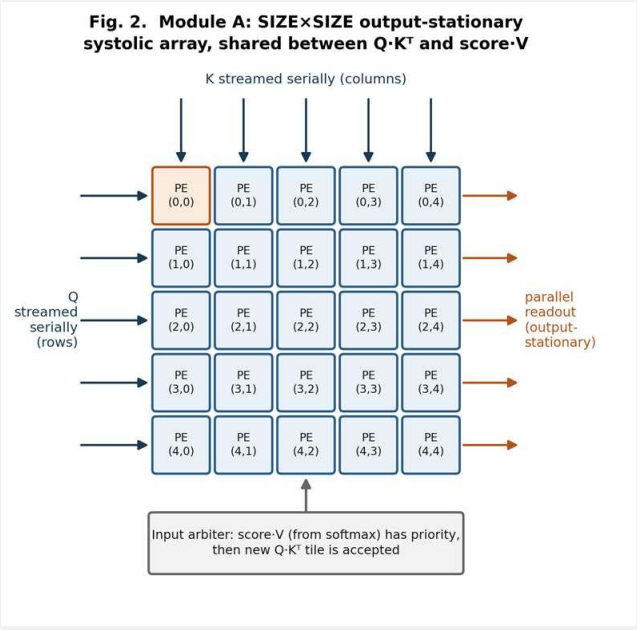


*Fig. 2. Structure of the SIZE×SIZE output-stationary systolic PE array (Module A), including the serial accumulation phase and the arbitrated input path shared between $Q \cdot K^T$ and score·V.*

## VIII. SCALING, MASKING, AND SOFTMAX

Raw similarity scores are scaled, and masked positions are set to negative infinity within a three-stage pipeline governed by a backpressure chain. Softmax itself is a four-state finite state machine: ACCUMULATE streams each row element through a LUT-based exponential unit and accumulates a running sum; once the row's last element has been summed, INVERT spends a single cycle turning that sum into a reciprocal through a second LUT. DIVIDE_NORMALIZE then streams the row's exponentials back out of a small FIFO, each one multiplied by that reciprocal, before the FSM returns to IDLE for the next row. This design trades throughput, one element at a time rather than a parallel reduction tree, for numerical accuracy from the LUT-based exponential and reciprocal.

## IX. MEMORY AND DATA ROUTING

The supporting infrastructure consists of the dbuf double-buffer, which allows new input data to be loaded while computation continues on the current tile, URAM-based memories that store the Q, K, and V vectors, and an output FIFO that transfers completed results to the AXI interface. The arbiters and routers that connect these blocks direct traffic on the shared hardware to its correct destination using the tag bit introduced in Section III.

## X. Verification and Resource Utilization

Every RTL module in this design has a standalone, self-checking testbench under tb/, summarized in Table I. All seven were executed to completion in Vivado's XSim behavioral simulator and pass their internal checks. Each is written to validate a single module or subsystem in isolation, so no module-level testbench requires the full top.sv pipeline to be assembled first.

TABLE I
TESTBENCH VERIFICATION SUMMARY

| Testbench | Covers | Result |
|---|---|---|
| tb_bf16_add.sv | Pipelined BF16 adder | PASS 7/7 |
| tb_bf16_mul.sv | Pipelined BF16 multiplier | PASS 6/6 |
| tb_gqa_mapper.sv | Q-head to KV-head mapping | PASS 8/8 |
| tb_qkv_proj.sv | Weight loading + W·x MAC | PASS 2/2 |
| tb_rope.sv | ROM load, rotate-half, sin/cos check | PASS |
| tb_qk_array.sv | Shared systolic array (mod_a_wrapper) | PASS 4/4 |
| tb_scale_mask_softmax.sv | Scale/mask/softmax, 2 rows | PASS 8/8 |

Running these testbenches surfaced concrete integration bugs that static review of four independently authored branches had not caught. The most severe was a single port declared 16 bits wide instead of 32 in axi_stream_if.sv, which let Vivado's synthesizer prove the entire output data path carried no observable information and delete nearly the whole design, from roughly 1,500 LUTs down to 73. Correcting that one port width restored the full pipeline. Other fixes included a missing write-address port on the shared URAM controller, an internally computed but never exposed m_last signal on the shared array's wrapper, and a tag-alignment mismatch between the softmax and shared systolic array interfaces.

A subsequent full top-level simulation, driving four tokens through the assembled pipeline end to end rather than exercising each module in isolation, surfaced four further integration bugs no module-level testbench could reach. dbuf.sv discarded every packet's final element because its bank-swap condition and its data-write condition were mutually exclusive branches of the same if/else, even though both were true on the same cycle. The URAM read controller advanced past its last requested address without confirming that address's result had been accepted downstream, silently losing it under backpressure. An arbiter's readiness signal back to softmax was wired behind a register that only became valid after softmax had already produced an output, a circular dependency that left softmax unable to produce its first result. And softmax's normalization multiplier lacked the single-element-in-flight guard already applied to the scale/mask stage, letting a later result overwrite an unconsumed earlier one. With all four fixed, the assembled design runs the full token-in to softmax-out path to completion without hanging or losing data; per-row softmax normalization remains open (Section XI).

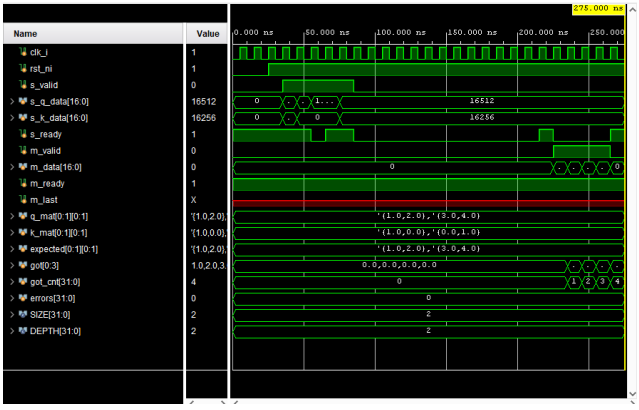


*Fig. 3. Waveform from tb_qk_array.sv, exercising the shared $Q \cdot K^T$ / score·V systolic array through mod_a_wrapper against a hand-computed 2×2 case (Q = [[1,2],[3,4]], identity K, so the expected result equals Q). All four output elements match, errors = 0. m_last reads X (shown in red) because this unit test does not connect that port. It is simply unexercised here, not a fault.*

Table II reports post-synthesis and post-implementation resource utilization on a Xilinx Artix-7 device (Nexys4 DDR, xc7a100tcsg324-1, Vivado 2024.1). The design uses no DSP48E1 slices and no Block RAM at all, consistent with the goal of a portable, LUT-only BF16 arithmetic core. The majority of LUT usage is distributed RAM for weight and ROM storage across the QKV projection and RoPE instances, rather than combinational logic.

TABLE II
POST-IMPLEMENTATION RESOURCE UTILIZATION

| Resource | Used | Available | % |
|---|---|---|---|
| Slice LUTs | 1,486 | 63,400 | 2.34 |
| Slice FFs | 510 | 126,800 | 0.40 |

| Resource | Used | Available | % |
|---|---|---|---|
| CARRY4 | 21 | 15,850 | 0.13 |
| DSP48E1 | 0 | 240 | 0.00 |
| Block RAM Tile | 0 | 135 | 0.00 |

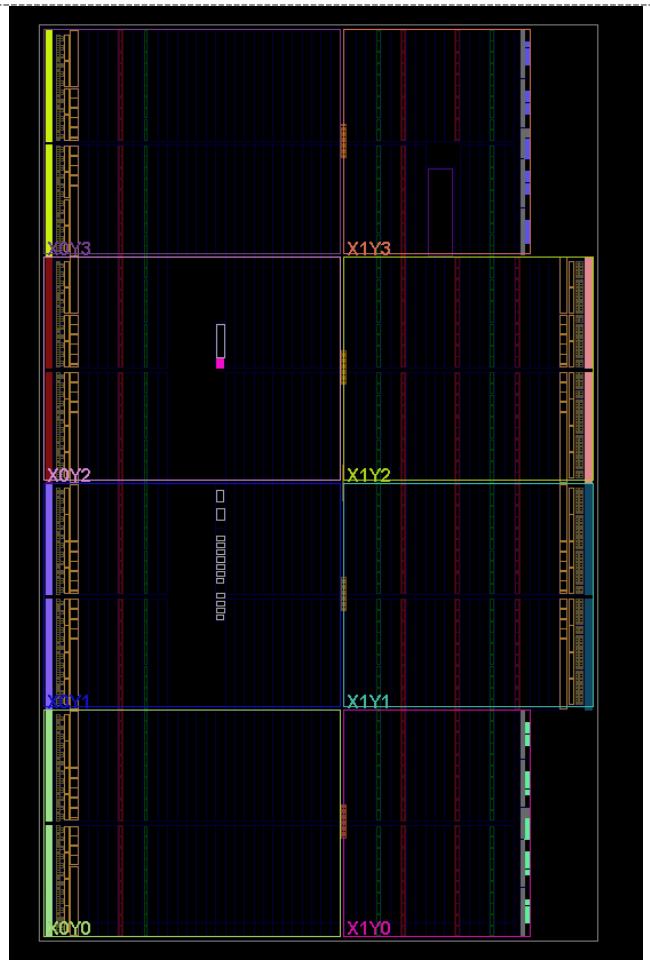


*Fig. 4. Post-implementation device view (clock regions X0Y0 to X1Y3, Nexys4 DDR / xc7a100t).*

## XI. OPEN ISSUES AND ASSUMPTIONS

The following items are still open, either flagged in code comments as pending team confirmation or identified as unfinished during integration:

- The RoPE rotate-half pairing rule (Section V) is implemented and exercised by tb_rope.sv, but not yet cross-checked against a Python/NumPy golden model across many positions.
- GQA head mapping (Section VI) produces a correct index but is not yet wired into the URAM controller's addressing, so the integrated design currently exercises a single head/KV-group path.
- The score·V tag alignment inside the shared array's input arbiter is a flagged assumption, not a confirmed team decision.
- No locking mechanism exists yet in the weight-loading process.
- A full end-to-end (top.sv) simulation has now been run to completion (four tokens, $Q \cdot K^T$ through softmax) without hanging or losing data, closing this item; multi-token throughput on real hardware, as opposed to simulation, remains unmeasured.

softmax currently normalizes the entire flattened score matrix as one row instead of one row per query, because the shared array's output drain asserts its end-of-row signal once per whole matrix rather than once per row. A one-line fix was attempted and reverted after it introduced a second, not-yet-diagnosed deadlock between softmax and the array drain. Timing closure and maximum clock frequency have not been measured.

## XII. CONCLUSION AND FUTURE WORK

The design presented here implements more than fifteen RTL modules covering the full self-attention pipeline, from projection through RoPE, GQA mapping, similarity computation, softmax, and output generation, with a shared systolic array reducing MAC resource duplication. Every module now has a passing standalone testbench, and the integration process that connected four independently written branches into a single working pipeline surfaced and fixed several concrete RTL bugs, most notably a port-width error that had caused the synthesizer to delete nearly the entire design (Section X). Post-implementation synthesis confirms a compact, DSP- and BRAM-free implementation at 1,486 LUTs and 510 flip-flops on a Xilinx Artix-7 device. A full end-to-end (top.sv) pipeline simulation has since been run to completion, surfacing and fixing four further integration bugs (Section X); planned next steps are correcting softmax's per-row normalization boundary (Section XI), cross-checking RoPE numerically against a golden software model, wiring GQA head selection into the URAM address path, adding a weight-loading handshake, and measuring timing closure and maximum clock frequency.

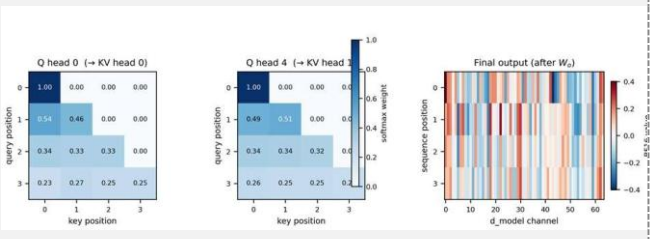


*Fig. 5. Golden-model reference output for a representative test case (seq_len=4, d_model=64, 8 Q / 2 KV heads, causal mask): softmax attention weights for two Q heads, and the final projected output, from the Python/BF16 golden model. Individual RTL stages now match this model's per-operation behavior at the module level (Section X). A matching full end-to-end (top.sv) RTL run against this exact reference is still pending.*

## REFERENCES


[1] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017.

[2] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv:2104.09864, 2021.

[3] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," arXiv:2305.13245, 2023.

[4] Google Brain, "bfloat16: The secret to high performance on Cloud TPUs," Google Cloud Blog, 2019.